

SERVICE ACTUEL · DEVSPACE-AGENT

Agent sortant de tunnel des devspaces

3 capacités analysées

services/devspace-agent · Python asyncio, websockets et Pydantic Settings.

Relie un devspace au coordinateur par WebSocket sortant et transporte les sessions vers son sshd local.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : Runtimes, Applications, IAM, Audit.

Périmètre de lecture

- Agent.run et canaux contrôle/données.
- Bridge WebSocket/TCP et configuration ; présence de tests, non exécutés.

Objets et cycle de vie

- Configuration agent, sessionId, connexion contrôle et tâches de pont ; état éphémère.

Dépendances du service

- Coordinateur de tunnel authentifié, sshd local, fichier de token projeté.

Fonctionnalités et compatibilité à préserver

DSA-01 Cible non qualifiée

Enregistrement sortant et ouverture de sessions

COMPORTEMENT OBSERVÉ

L'agent ouvre un WebSocket vers agent/connect avec un bearer lu sur disque. Les messages de contrôle JSON de type session_request créent une tâche par sessionId ; les frames binaires ou mal formées sont ignorées. La session ouvre le sshd local puis un WebSocket agent/session/{sessionId}. Le token est relu à chaque connexion, ce qui permet de prendre en compte une rotation du fichier. Le code lu ne déduplique pas les sessionId avant de créer une tâche.

UTILISATEURS ET RÔLES

Devspace et coordinateur de tunnels.

INTERFACES DANS LE CODE

Agent._connect_and_serve ; Agent._serve_control_channel ; Agent._handle_session.

DONNÉES ET ÉTATS

Connexion de contrôle et ensemble de tâches locales ; aucune base persistante.

DÉPENDANCES ET ÉCHANGES

Coordinateur WSS, token projeté et sshd local.

COMPORTEMENT CIBLE PROPOSÉ

Runtimes/Applications Rust peut reprendre l'agent en Rust ou encapsuler le composant sans JVM transitoirement ; session, identité et durée sont explicitement bornées.

MIGRATION DE CETTE CAPACITÉ

Préserver le protocole binaire et les URLs durant migration du coordinateur ; ajouter limites de sessions et déduplication de demande.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Renvoyer deux fois la même demande ne crée pas deux tunnels logiques. Après rotation du token, la reconnexion utilise la nouvelle identité sans redémarrer le pod.

Références de l'analyse

[monorepo](#) · [services/devspace-agent/src/devspace_agent/agent.py:61](#) Enregistrement authentifié.

[monorepo](#) · [services/devspace-agent/src/devspace_agent/agent.py:74](#) Validation des frames et création de tâches.

[monorepo](#) · [services/devspace-agent/src/devspace_agent/agent.py:127](#) Relecture du token.

DSA-02 Cible non qualifiée

Transport des octets et fermeture du tunnel**COMPORTEMENT OBSERVÉ**

Le bridge lance deux pompes asyncio : frames WebSocket vers TCP avec drain, et lectures TCP par chunks vers WebSocket. Les frames textuelles sont encodées en octets ; les payloads vides sont ignorés. La fin d'une pompe entraîne l'annulation de l'autre puis la fermeture du writer TCP. Le code ne parse ni commande SSH ni contenu métier. Les connexions WebSocket emploient `max_size=None` dans l'agent ; des plafonds globaux doivent donc être évalués au niveau du tunnel.

UTILISATEURS ET RÔLES

Client SSH et sshd du devspace.

INTERFACES DANS LE CODE

bridge ; ws_to_tcp ; tcp_to_ws.

DONNÉES ET ÉTATS

Buffers et tâches éphémères ; flux chiffré SSH relayé comme octets.

DÉPENDANCES ET ÉCHANGES

Asyncio streams et WebSocket du coordinateur.

COMPORTEMENT CIBLE PROPOSÉ

Runtimes Rust offre un transport à backpressure avec budgets d'octets, durée et connexions, sans inspection métier du flux SSH.

MIGRATION DE CETTE CAPACITÉ

Qualifier EOF, fermeture distante, annulation et débit avant remplacement. Ne pas déduire la sémantique de half-close du seul commentaire : vérifier le comportement effectif.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un transfert binaire long conserve l'intégrité et borne la mémoire quand le lecteur ralentit. Une fermeture ou annulation libère les tâches et sockets sans tuer un autre tunnel.

Références de l'analyse

[monorepo](#) · [services/devspace-agent/src/devspace_agent/bridge.py:25](#) Pompes bidirectionnelles et fermeture.

[monorepo](#) · [services/devspace-agent/src/devspace_agent/agent.py:69](#) Absence de limite de frame dans les connexions lues.

DSA-03 **Cible non qualifiée****Reconnexion, arrêt et paramètres de l'agent****COMPORTEMENT OBSERVÉ**

La boucle reconnecte après erreur avec backoff exponentiel plafonné et jitter ; une connexion réussie réinitialise le délai. L'arrêt positionne un événement et annule les tâches de sessions connues. La configuration porte l'URL du coordinateur, la cible sshd, le chemin du token, les délais de reconnexion et les paramètres de healthcheck. Aucun journal de session durable n'est conservé dans ce composant.

UTILISATEURS ET RÔLES

Opérateur et orchestrateur du devspace.

INTERFACES DANS LE CODE

Agent.run ; Agent.stop ; AgentConfig.

DONNÉES ET ÉTATS

Backoff, événement d'arrêt, ensemble des tâches et configuration du processus.

DÉPENDANCES ET ÉCHANGES

Cycle de vie du pod, coordinateur et fichier de token.

COMPORTEMENT CIBLE PROPOSÉ

Applications/Runtimes Rust observe enregistrement, sessions actives et refus, avec reconnexion bornée et arrêt drainé. Audit journalise les ouvertures/fermetures côté autorité sans contenu SSH.

MIGRATION DE CETTE CAPACITÉ

Conserver la compatibilité des paramètres publics et mesurer les tempêtes de reconnexion ; décider explicitement les limites de durée et d'inactivité.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Redémarrer le coordinateur pour plusieurs devspaces ne crée pas une boucle serrée. L'arrêt ne laisse pas de session active sans propriétaire identifié.

Références de l'analyse

[monorepo](#) · [services/devspace-agent/src/devspace_agent/agent.py:35](#) Backoff et reprise.

[monorepo](#) · [services/devspace-agent/src/devspace_agent/agent.py:56](#) Arrêt et annulation des sessions.

[monorepo](#) · [services/devspace-agent/src/devspace_agent/config.py:4](#) Paramètres du processus.

Architecture et exploitation du service cible**Plan de contrôle Rust proposé**

- Autoriser chaque ouverture et révocation côté coordinateur ; bornes de durée et sessions.

Plan de données proposé

- Transport opaque WebSocket/TCP avec backpressure et ressources bornées.

Découpage du core Rust

- Agent Rust asynchrone et bibliothèque de bridge ; protocole compatible pendant transition.

Identité, autorisation et isolation

- Token relu sans être imprimé ; le tunnel transporte un accès SSH puissant à limiter au bon devspace.
- Les limites de frame/session doivent être explicites ; max_size=None est un état du code lu.

Exploitation et reprise

- Backoff avec jitter ; supervision enregistrement et sessions.
- Aucune donnée métier durable dans l'agent ; restauration par reconnexion.

Séquence de migration

- Qualifier protocole/EOF et token rotation.
- Canary d'un agent Rust avec même coordinateur.
- Remplacer les images et retirer le runtime Python de cet agent si la politique le prévoit.

Écarts et décisions encore ouverts

- La qualification de charge, half-close et révocation côté coordinateur n'a pas été exécutée.
- Aucun plafond de sessions actives constaté dans Agent._spawn_session.

Références de l'analyse

[monorepo](#) · [services/devspace-agent/src/devspace_agent/agent.py:29](#) Lifecycle agent.

[monorepo](#) · [services/devspace-agent/src/devspace_agent/bridge.py:25](#) Transport.